Pimpri-Chinchwad Education Trust's
Pimpri-Chinchwad College of Engineering
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU)
ISO 21001:2018 Certified by TUV

Assignment 4

Name : Om Rajendra Bharambe
PRN : 124B1F001
Batch : A1
Date : 10/03/2026

Aim:

Write code to implement **Procedure and Function**.

Objectives:

- **To implement modular database logic** by creating stored procedures that encapsulate complex CRUD operations, ensuring reusable and maintainable code across the Airline and Civic Voice systems.
- **To develop custom computational logic** using SQL functions to perform dynamic calculations, such as determining flight revenue, total user spending, and document verification metrics.
- **To enhance data security and abstraction** by utilizing procedures to manage sensitive data operations, allowing users to interact with the database through controlled interfaces rather than direct table manipulation.
- **To optimize query execution** by leveraging stored routines that reside on the database server, reducing network traffic and improving the performance of repetitive tasks like status updates and statistical reporting.

Case Study: Airline Management System

SQL Queries:

1. Environment Setup and Database Selection

The following commands initialize the session, show the available databases, and select the airlines database for operation.

```
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 8.0.45 MySQL Community Server - GPL

mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| airlines           |
| banksystem         |
| civic_voice        |
| civic_voice_trig   |
| fkpk               |
| information_schema |
| mysql              |
| performance_schema |
| sakila             |
| sqldetective       |
| sys                |
| world              |
+--------------------+
12 rows in set (0.00 sec)

mysql> USE airlines;
Database changed
```

---

2. Defining Flight Management Stored Procedures

These procedures handle the **CRUD** (Create, Read, Update, Delete) operations for the *flight* table.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE AddFlight(IN p_id INT, IN p_name VARCHAR(50), IN p_src VARCHAR(50),
IN p_dest VARCHAR(50), IN p_price DECIMAL(10,2), IN p_airline_id INT)
    -> BEGIN
    -> INSERT INTO flight(flight_id, flight_name, source, destination, price, airline_id)
VALUES(p_id, p_name, p_src, p_dest, p_price, p_airline_id);
    -> END //
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE PROCEDURE UpdateFlightPrice(IN p_id INT, IN new_price DECIMAL(10,2))
    -> BEGIN
    -> UPDATE flight SET price = new_price WHERE flight_id = p_id;
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE PROCEDURE DeleteFlight(IN p_id INT)
    -> BEGIN
    -> DELETE FROM flight WHERE flight_id = p_id;
    -> END //
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE PROCEDURE ShowFlights()
    -> BEGIN
    -> SELECT * FROM flight;
    -> END //
Query OK, 0 rows affected (0.00 sec)
```

---

3. Defining Flight Analysis Functions

These functions calculate revenue, retrieve specific flight prices, and count total flights within the system.

```
mysql> CREATE FUNCTION GetFlightRevenue(p_flight_id INT) RETURNS DECIMAL(12,2) DETERMINISTIC
    -> BEGIN
    -> DECLARE total_revenue DECIMAL(12,2);
    -> DECLARE f_price DECIMAL(10,2);
    -> DECLARE b_count INT;
    -> SELECT price INTO f_price FROM flight WHERE flight_id = p_flight_id;
    -> SELECT COUNT(*) INTO b_count FROM booking WHERE flight_id = p_flight_id;
    -> SET total_revenue = f_price * b_count;
    -> RETURN IFNULL(total_revenue, 0.00);
    -> END //
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE FUNCTION GetFlightPrice(p_flight_id INT) RETURNS DECIMAL(10,2) DETERMINISTIC
    -> BEGIN
    -> DECLARE f_price DECIMAL(10,2);
    -> SELECT price INTO f_price FROM flight WHERE flight_id = p_flight_id;
    -> RETURN f_price;
    -> END //
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE FUNCTION TotalFlights() RETURNS INT DETERMINISTIC
    -> BEGIN
    -> DECLARE total INT;
    -> SELECT COUNT(*) INTO total FROM flight;
    -> RETURN total;
    -> END //
Query OK, 0 rows affected (0.00 sec)

mysql> DELIMITER ;
```

4. Execution and Testing: Flight Operations

The following logs demonstrate the execution of the stored procedures and functions defined above.

```
mysql> CALL AddFlight(105, 'AI105', 'Delhi', 'Mumbai', 4800.00, 1);
Query OK, 1 row affected (0.00 sec)

mysql> SELECT * FROM flight;
+-----------+-------------+--------+-------------+----------+------------+
| flight_id | flight_name | source | destination | price    | airline_id |
+-----------+-------------+--------+-------------+----------+------------+
|       101 | AI101       | Mumbai | Delhi       |  5500.00 |          1 |
|       102 | IN202       | Pune   | Bangalore   |  4500.00 |          2 |
|       103 | SJ303       | Delhi  | Goa         |  6000.00 |          3 |
|       104 | EM404       | Dubai  | Mumbai      | 15000.00 |          4 |
|       105 | AI105       | Delhi  | Mumbai      |  4800.00 |          1 |
+-----------+-------------+--------+-------------+----------+------------+
5 rows in set (0.00 sec)

mysql> CALL UpdateFlightPrice(105, 5200.00);
Query OK, 1 row affected (0.00 sec)

mysql> SELECT GetFlightRevenue(101) AS Revenue_Flight_101;
+--------------------+
| Revenue_Flight_101 |
+--------------------+
|            5500.00 |
+--------------------+
1 row in set (0.00 sec)

mysql> SELECT TotalFlights() AS Total_Flights_In_System;
+-------------------------+
| Total_Flights_In_System |
+-------------------------+
|                       5 |
+-------------------------+
1 row in set (0.00 sec)
```

---

5. Defining Passenger and Booking Operations

This section includes procedures for adding passengers and managing bookings, as well as functions for financial tracking.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE AddPassenger(IN p_id INT, IN p_name VARCHAR(50), IN p_gender
CHAR(1), IN p_age INT, IN p_phone VARCHAR(15))
    -> BEGIN
    -> INSERT INTO passenger(passenger_id, pname, gender, age, phone) VALUES(p_id, p_name,
p_gender, p_age, p_phone);
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE PROCEDURE CreateBooking(IN b_id INT, IN p_id INT, IN f_id INT, IN s_no
VARCHAR(5))
    -> BEGIN
    -> INSERT INTO booking(booking_id, passenger_id, flight_id, booking_date, seat_no)
VALUES(b_id, p_id, f_id, CURDATE(), s_no);
    -> END //
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE FUNCTION GetTotalSpent(p_pass_id INT) RETURNS DECIMAL(12,2) DETERMINISTIC
    -> BEGIN
    -> DECLARE total DECIMAL(12,2);
    -> SELECT SUM(f.price) INTO total FROM booking b JOIN flight f ON b.flight_id =
f.flight_id WHERE b.passenger_id = p_pass_id;
    -> RETURN IFNULL(total, 0.00);
    -> END //
Query OK, 0 rows affected (0.00 sec)

mysql> DELIMITER ;
```

6. Execution and Testing: Passenger and Booking Records

Final verification of data insertion, booking creation, and deletion.

```
mysql> CALL AddPassenger(5, 'Vikram Singh', 'M', 40, '9876501234');
Query OK, 1 row affected (0.00 sec)

mysql> CALL CreateBooking(5, 5, 102, '15C');
Query OK, 1 row affected (0.00 sec)

mysql> SELECT * FROM booking;
+------------+--------------+-----------+--------------+---------+
| booking_id | passenger_id | flight_id | booking_date | seat_no |
+------------+--------------+-----------+--------------+---------+
|          1 |            1 |       101 | 2026-03-04   | 12A     |
|          2 |            2 |       102 | 2026-03-04   | 14B     |
|          4 |            4 |       104 | 2026-03-04   | 8D      |
|          5 |            5 |       102 | 2026-04-14   | 15C     |
+------------+--------------+-----------+--------------+---------+
4 rows in set (0.00 sec)

mysql> SELECT GetTotalSpent(5) AS Vikram_Total_Spent;
+--------------------+
| Vikram_Total_Spent |
+--------------------+
|            4500.00 |
+--------------------+
1 row in set (0.00 sec)

mysql> CALL CancelBooking(5);
Query OK, 1 row affected (0.00 sec)
```

Case Study: Civic Voice Interface System

SQL Queries:

1. Database Initialization and Selection

This section shows the initial environment setup, listing available databases and selecting the active workspace.

```
mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| airlines           |
| banksystem         |
| civic_voice        |
| civic_voice_trig   |
| fkpk               |
| information_schema |
| mysql              |
| performance_schema |
| sakila             |
| sqldetective       |
| sys                |
| world              |
+--------------------+
12 rows in set (0.00 sec)

mysql> USE civic_voice;
Database changed
```

---

2. Complaint Management Stored Procedures

The following procedures define the logic for adding, updating, deleting, and viewing user complaints.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE AddComplaint(IN c_id INT, IN u_id INT, IN c_title VARCHAR(100), IN
c_desc TEXT)
    -> BEGIN
    -> INSERT INTO complaint(complaint_id, user_id, title, description) VALUES(c_id, u_id,
c_title, c_desc);
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE PROCEDURE UpdateComplaintStatus(IN c_id INT, IN new_status VARCHAR(20))
    -> BEGIN
    -> UPDATE complaint SET status = new_status WHERE complaint_id = c_id;
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE PROCEDURE DeleteComplaint(IN c_id INT)
    -> BEGIN
    -> DELETE FROM complaint WHERE complaint_id = c_id;
    -> END //
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE PROCEDURE ShowUserComplaints(IN u_id INT)
    -> BEGIN
    -> SELECT * FROM complaint WHERE user_id = u_id;
    -> END //
Query OK, 0 rows affected (0.00 sec)
```

---

3. Data Retrieval and Analysis Functions

These functions are used to calculate metrics such as document counts, verification status, and the total number of complaints in the system.

```
mysql> CREATE FUNCTION GetUserDocumentCount(p_user_id INT) RETURNS INT DETERMINISTIC
    -> BEGIN
    -> DECLARE doc_count INT;
    -> SELECT COUNT(*) INTO doc_count FROM userdocument WHERE user_id = p_user_id;
    -> RETURN doc_count;
    -> END //
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE FUNCTION GetTotalVerifiedDocs(p_user_id INT) RETURNS INT DETERMINISTIC
    -> BEGIN
    -> DECLARE verified_count INT;
    -> SELECT COUNT(*) INTO verified_count FROM userdocument WHERE user_id = p_user_id AND
is_verified = 1;
    -> RETURN verified_count;
    -> END //
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE FUNCTION TotalComplaints() RETURNS INT DETERMINISTIC
    -> BEGIN
    -> DECLARE total INT;
    -> SELECT COUNT(*) INTO total FROM complaint;
    -> RETURN total;
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql> DELIMITER ;
```

---

4. Execution and Verification: Complaint Workflow

This log demonstrates the practical application of the procedures: adding a complaint, updating its status, and retrieving the record.

```
mysql> CALL AddComplaint(1, 1, 'Pothole Issue', 'Severe pothole on MG Road causing traffic');
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM complaint;
+--------------+---------+--------------+-----------------------------------------+---------+--------------+
| complaint_id | user_id | title        | description                             | status  | created_date |
+--------------+---------+--------------+-----------------------------------------+---------+--------------+
|            1 |       1 | Pothole Issue | Severe pothole on MG Road causing traffic | Pending | 2026-04-15   |
+--------------+---------+--------------+-----------------------------------------+---------+--------------+
1 row in set (0.00 sec)

mysql> CALL UpdateComplaintStatus(1, 'In Progress');
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM complaint WHERE complaint_id = 1;
+--------------+---------+--------------+-----------------------------------------+------------+--------------+
| complaint_id | user_id | title        | description                             | status     | created_date |
+--------------+---------+--------------+-----------------------------------------+------------+--------------+
|            1 |       1 | Pothole Issue | Severe pothole on MG Road causing traffic | In Progress | 2026-04-15   |
+--------------+---------+--------------+-----------------------------------------+------------+--------------+
1 row in set (0.00 sec)
```

---

5. Execution and Verification: Functional Queries and Cleanup

Final checks using functions to retrieve statistics, followed by the deletion of the test complaint record.

```
mysql> SELECT GetUserDocumentCount(1) AS Rahul_Total_Docs;
+------------------+
| Rahul_Total_Docs |
+------------------+
|                2 |
+------------------+
1 row in set (0.00 sec)

mysql> SELECT GetTotalVerifiedDocs(1) AS Rahul_Verified_Docs;
+---------------------+
| Rahul_Verified_Docs |
+---------------------+
|                   2 |
+---------------------+
1 row in set (0.00 sec)
```

```
mysql> SELECT TotalComplaints() AS Total_Complaints_In_System;
+---------------------------+
| Total_Complaints_In_System |
+---------------------------+
|                         1 |
+---------------------------+
1 row in set (0.00 sec)

mysql> CALL DeleteComplaint(1);
Query OK, 1 row affected (0.00 sec)

mysql> SELECT * FROM complaint;
Empty set (0.00 sec)
```

Conclusion:

Through this assignment, the fundamental concepts of **Stored Procedures** and **User-Defined Functions** were successfully implemented. These PL/SQL (or MySQL procedural) blocks demonstrate how to move application logic directly into the database tier. By using procedures like AddFlight and UpdateComplaintStatus, we achieved a higher degree of data consistency. Similarly, functions like GetFlightRevenue and GetTotalVerifiedDocs showcased the power of returning specific values based on complex joins and aggregations, providing a robust framework for backend automation and data integrity.